

Machine Learning-Based Anomaly Detection for Traffic in IoT-Enabled Transportation Networks

Connor Gladish¹, Molly Corgan¹, Jason Moss¹ , Blanca Negrete¹, Lily J. Pharris¹ , Qing Wang¹ , Charlie Liu², Yongxin Liu³  and Jian Wang^{1,*} 

¹ Department of Computer Science at the University of Tennessee at Martin;

² Department of Mathematics at the University of Tennessee at Martin;

³ Department of Mathematics at the Embry-Riddle Aeronautical University;

* Correspondence: jwang186@utm.edu;

Abstract

With the rapid expansion of Internet of Things (IoT) technologies in modern transportation systems, network connectivity has become fundamental to enabling real-time communication, automation, and data-driven decision-making. However, increased connectivity also expands the attack surface, making transportation networks more vulnerable to cyber threats, malicious traffic, and anomalous behavior. This project focuses on detecting anomalies within IP-based transportation network traffic using machine learning classification techniques. Leveraging publicly available datasets, including labeled IP network traffic flows across multiple applications, we analyze traffic patterns to distinguish between normal and suspicious behavior. The research evaluates several lightweight classification models to determine their effectiveness based on detection accuracy, computational efficiency, and adaptability to transportation-specific network environments. By comparing model performance and analyzing misclassifications, the project aims to identify practical approaches for real-time anomaly detection in resource-constrained IoT transportation systems. This work contributes to improving the security and resilience of connected transportation infrastructures by providing a scalable framework for network traffic analysis and threat detection.

Keywords: IoT security; intrusion detection; anomaly detection; connected vehicles; V2X; machine learning; XGBoost; network traffic analysis

Received:

Revised:

Accepted:

Published:

Citation: Gladish, C.; Corgan, M.; Moss, J.; Negrete, B.; Pharris, L.; Wang, Q.; Liu, C.; Liu, Y.; & Wang, J. Machine Learning-Based Anomaly Detection for Traffic in IoT-Enabled Transportation Networks. *Journal Not Specified* **2026**, *1*, 0. <https://doi.org/>

Copyright: © 2026 by the authors. Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Modern transportation infrastructures are rapidly evolving into highly connected cyber-physical systems that rely on continuous communication between vehicles, roadside infrastructure, and cloud-based services. Through the integration of embedded sensors, wireless communication protocols, and distributed computing platforms, intelligent transportation infrastructures now enable real-time data collection [1], vehicle-to-vehicle (V2V) communication [2], and automated decision-making [3]. These capabilities support advanced applications such as traffic optimization, autonomous driving, predictive maintenance, and smart mobility services [4]. As transportation networks continue to evolve toward fully connected and autonomous environments, reliable and secure data exchange has become a critical requirement for maintaining system efficiency and public safety.

Despite these advancements, the increasing reliance on interconnected IoT devices introduces substantial cybersecurity challenges. Transportation systems built on distributed

networks and diverse communication protocols significantly expand the potential attack surface. Malicious actors may exploit vulnerabilities within networked components to launch attacks such as distributed denial-of-service (DDoS), spoofing, or manipulation of sensor data within V2V communication channels [5]. These attacks can disrupt network functionality, compromise system integrity, and potentially lead to dangerous real-world consequences, including incorrect vehicle behavior or failures in traffic management systems [5]. Prior work has demonstrated that IoT-enabled transportation infrastructure is susceptible to cyber-physical attacks at the sensing layer due to inherent vulnerabilities of embedded IoT devices [6]. Malware targeting IoT-enabled intelligent transportation systems represents a particularly active threat vector, as recent work has shown that deep learning techniques can be applied to detect such attacks with high accuracy [7].

Traditional network security approaches, including rule-based intrusion detection systems and static filtering techniques, often struggle to address the dynamic and high-volume nature of IoT transportation traffic. These systems typically rely on predefined attack signatures and therefore lack the flexibility to detect novel or evolving threats [8]. Furthermore, IoT transportation environments often involve resource-constrained devices with limited computational capacity, making it challenging to deploy complex security mechanisms without affecting system performance [8]. The challenge is compounded in maritime and other specialized transportation domains, where IoT devices are resource-constrained and traditional offline or batch-based detection systems are difficult to deploy effectively [9].

Machine learning (ML) has emerged as a promising solution for addressing these limitations by enabling adaptive and data-driven anomaly detection. ML-based classification models can analyze large volumes of network traffic data to identify patterns associated with both normal and malicious behavior. By learning from labeled datasets, these models can detect previously unseen attack patterns while maintaining efficient operation suitable for real-time environments. In the context of intelligent transportation systems, lightweight ML approaches offer the potential to improve threat detection while remaining compatible with the computational limitations of embedded IoT devices. Recent evaluations of anomaly detection methods in intelligent transportation systems have highlighted the importance of selecting approaches that balance detection effectiveness with computational practicality [10]. ML methods applied to transportation traffic data must also contend with class imbalance, as labeled datasets frequently contain large volumes of common traffic classes alongside very few samples of rare attack types [11]. Prior work on intrusion detection in IoT transportation networks has also been noted that deep learning models can produce high false-positive rates and that the interpretability of model decisions is an important practical concern [12].

This research investigates the application of machine learning classification techniques for anomaly detection in IoT-enabled transportation networks. Using publicly available datasets containing labeled IP-based network traffic flows across multiple application types, the project analyzes traffic characteristics to distinguish between benign and suspicious network behavior. Several classification models are evaluated to assess their effectiveness in terms of detection accuracy, computational efficiency, and adaptability to transportation-specific networking conditions. By identifying practical approaches for detecting anomalous traffic patterns, this work continues to improve the cybersecurity resilience of connected transportation infrastructures. The results support the development of scalable security solutions capable of protecting next-generation intelligent transportation systems while maintaining operational efficiency.

The remainder of this paper is organized as follows. Section 2 describes the dataset, preprocessing pipeline, and model configurations. Section 3 reports binary and multi-class

results, Receiver Operating Characteristic (ROC) analysis, feature importance, and per-class performance. Section 4 presents an error analysis across all five models and discusses deployment implications. Section 5 concludes the paper and outlines future work.

2. Methodology

2.1. Dataset

We evaluate all models on the ACI-IoT-2023 dataset, released by the Army Cyber Institute [13]. The raw dataset contains 1,231,411 network flow records, each described by 78 numerical and categorical features extracted from flow-level statistics. The label space contains eleven retained classes: Port Scan, Benign, ICMP Flood, Ping Sweep, DNS Flood, Vulnerability Scan, OS Scan, Slowloris, SYN Flood, Dictionary Attack, and UDP Flood. The class distribution is highly imbalanced, as shown in Table 1.

Table 1. ACI-IoT-2023 class distribution after rare-class filtering.

Class	Samples	Percentage
Port Scan	441,282	35.80%
Benign	329,295	26.70%
ICMP Flood	225,234	18.30%
Ping Sweep	71,928	5.80%
DNS Flood	46,935	3.80%
Vulnerability Scan	39,537	3.20%
OS Scan	37,524	3.00%
Slowloris	18,643	1.50%
SYN Flood	13,857	1.10%
Dictionary Attack	6,380	0.50%
UDP Flood	791	0.06%

Port Scan alone accounts for more than one-third of all records, while UDP Flood comprises only 0.06% of the dataset. This imbalance has direct consequences for minority-class detection, as discussed in Section 4.

2.2. Preprocessing Pipeline

The preprocessing pipeline is designed to prevent data leakage, preserve proportional class representation across splits, and enable controlled comparison across all five algorithms. It consists of the following nine steps.

1) Data ingestion and rare-class removal. Classes with fewer than 100 total samples (specifically ARP Spoofing) are removed, leaving eleven reliably sampled classes.

2) Label engineering. Two parallel label arrays are constructed: an integer-encoded multi-class array (0–10) and a binary array (0 = benign, 1 = attack). Non-predictive identifier columns such as IP addresses, timestamps, and flow IDs are removed at this stage.

3) Categorical encoding and cleaning. All categorical columns are integer-encoded independently using `LabelEncoder`. NaN values are imputed with the column-wise mean; infinite values, which arise when flow denominators approach zero, are replaced with NaN and then filled with zero. Final feature dimensionality is approximately 47–50 features.

4) Stratified downsampling. Because training K-Nearest Neighbors (KNN) and Random Forest on more than one million records is computationally prohibitive, we draw a stratified sample of exactly 500,000 records with `random_state=42`, preserving each class’s proportional representation.

5) Train/validation/test split. Two sequential stratified splits produce approximately 350,000 training, 75,000 validation, and 75,000 test records (70/15/15). The test set is isolated before any model fitting and accessed exactly once, at final evaluation.

6) Feature scaling. All features are standardized to zero mean and unit variance. The `StandardScaler` is fit exclusively on the training set and applied—without refitting—to the validation and test sets. Although tree-based models are scale-invariant, uniform scaling ensures that KNN distance computations, eXtreme Gradient Boosting (XGBoost) gradient updates, and Convolutional Neural Network (CNN) gradient flow all operate on comparable feature magnitudes, so that observed performance differences reflect algorithmic properties rather than preprocessing artifacts.

7–8) Training. All five classifiers are trained on the binary-labeled training set and subsequently retrained on the multi-class labels. For the multi-class task, XGBoost switches to `multi:softmax` with `num_class=11`, and the CNN output layer changes from a single sigmoid unit to an 11-unit softmax. All other hyperparameters remain identical to enable controlled comparison.

9) Evaluation. For each model and task we compute accuracy, weighted precision, weighted recall, weighted F1-score, AUC-ROC (binary), per-class classification reports, and wall-clock training time, all evaluated on the held-out test set.

2.3. Models

Five classifiers with distinct inductive biases are evaluated:

Decision Tree. A deterministic depth-20 tree trained with scikit-learn’s default Gini-impurity splitting criterion. It creates explicit, inspectable axis-aligned decision rules.

Random Forest. An ensemble of 100 bootstrapped depth-20 trees with majority voting. Bagging is expected to improve calibration over a single tree at the cost of slower training.

XGBoost. A gradient-boosted ensemble of 100 trees with `max_depth=10` and `learning_rate=0.1` [14]. Sequential boosting focuses learning capacity on previously misclassified examples.

KNN. A lazy learner with $k = 5$ using Euclidean distance. KNN has no training phase; classification cost is incurred at inference time.

1D CNN. A deep model that treats the 78 flow features as a length-78 sequence and applies three `Conv1D` blocks with kernel size 3 (64, 128, and 64 filters), each followed by batch normalization and ReLU activation. Two `MaxPooling1D` layers reduce the spatial dimension from 78 to 19. A `GlobalAveragePooling1D` layer collapses the sequence dimension, followed by two `Dense` layers (128 and 64 units) with `Dropout(0.3)`. The output layer is a single sigmoid unit for binary classification or an 11-unit softmax for multi-class classification. The network has 67,265 parameters (binary) or 67,915 parameters (multi-class). Training uses early stopping with a small train/validation loss gap throughout, indicating that the architecture—not overfitting—limits performance.

3. Results

3.1. Binary Classification

Table 2 summarizes binary classification performance on the held-out test set of approximately 75,000 records. Each model’s confusion matrix is presented individually below alongside a discussion of its classification behavior.

Decision Tree. As shown in Fig. 1, the Decision Tree correctly classifies 26.64% of flows as benign and 73.13% as attacks, with a false-positive rate of only 0.10% and a false-negative rate of 0.13%. It achieves the highest F1-score of all five models (99.85%) and the fewest false positives overall. Its axis-aligned splitting on dominant features such as RST Flag

Table 2. Binary classification results (benign vs. attack).

Model	Acc.	Prec.	Rec.	F1	AUC	Time (min)
Decision Tree	99.77%	99.87%	99.83%	99.85%	0.9968	0.17
Random Forest	99.72%	99.79%	99.83%	99.81%	0.9999	0.31
XGBoost	99.74%	99.82%	99.83%	99.82%	0.9999	0.08
KNN	99.57%	99.70%	99.71%	99.70%	0.9980	—
1D CNN	99.24%	99.43%	99.53%	99.48%	0.9990	1.25

Count allows it to route the vast majority of flows correctly in a single deterministic pass, with no ensemble overhead.

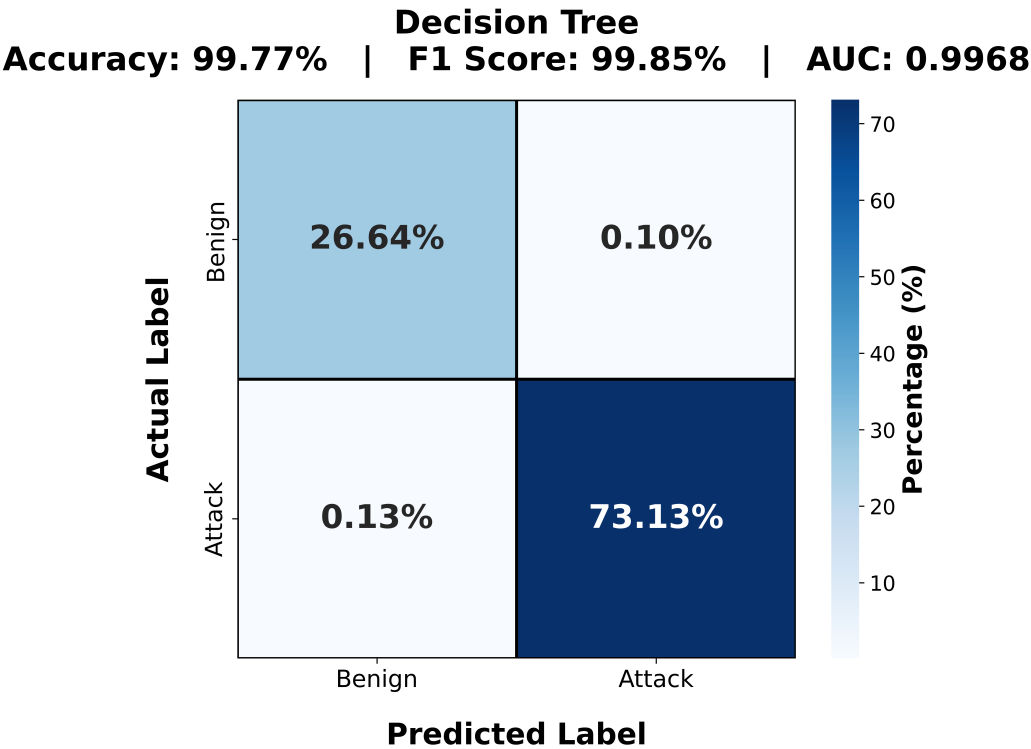


Figure 1. Decision Tree binary confusion matrix.

Random Forest. The Random Forest (Fig. 2) produces nearly identical cell counts to the Decision Tree—26.59% true benign, 73.13% true attack—but with a slightly higher false-positive rate of 0.15%. It ties XGBoost for the best AUC-ROC (0.9999), indicating near-perfect probability calibration across all thresholds. The marginal increase in false positives relative to the Decision Tree reflects ensemble voting behavior, where slim attack majorities on borderline benign flows override what a single tree would route correctly.

XGBoost. XGBoost (Fig. 3) achieves 26.61% true benign and 73.13% true attack, with a false-positive rate of 0.13% and a false-negative rate of 0.12%—the lowest false-negative rate among all five models. It matches Random Forest on AUC-ROC (0.9999) while training in only 0.08 min, making it approximately 4× faster than the Decision Tree and 15× faster than the CNN. This combination of near-perfect discrimination and minimal training time makes XGBoost the strongest candidate for the first stage of a two-stage deployment pipeline.

KNN. The KNN classifier (Fig. 4) correctly identifies 26.52% of flows as benign and 73.04% as attacks. Its false-positive rate of 0.22% and false-negative rate of 0.21% are the highest among the classical models, reflecting the known sensitivity of uniform-distance KNN to low-importance features diluting the signal from dominant ones such as RST Flag

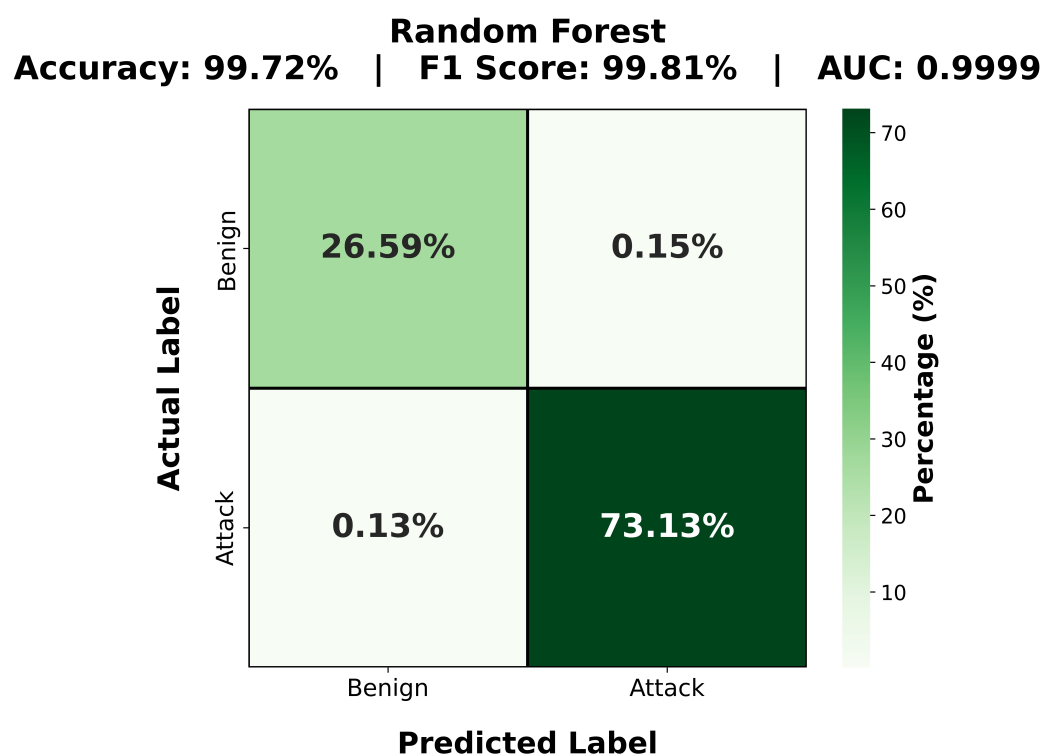


Figure 2. Random Forest binary confusion matrix.

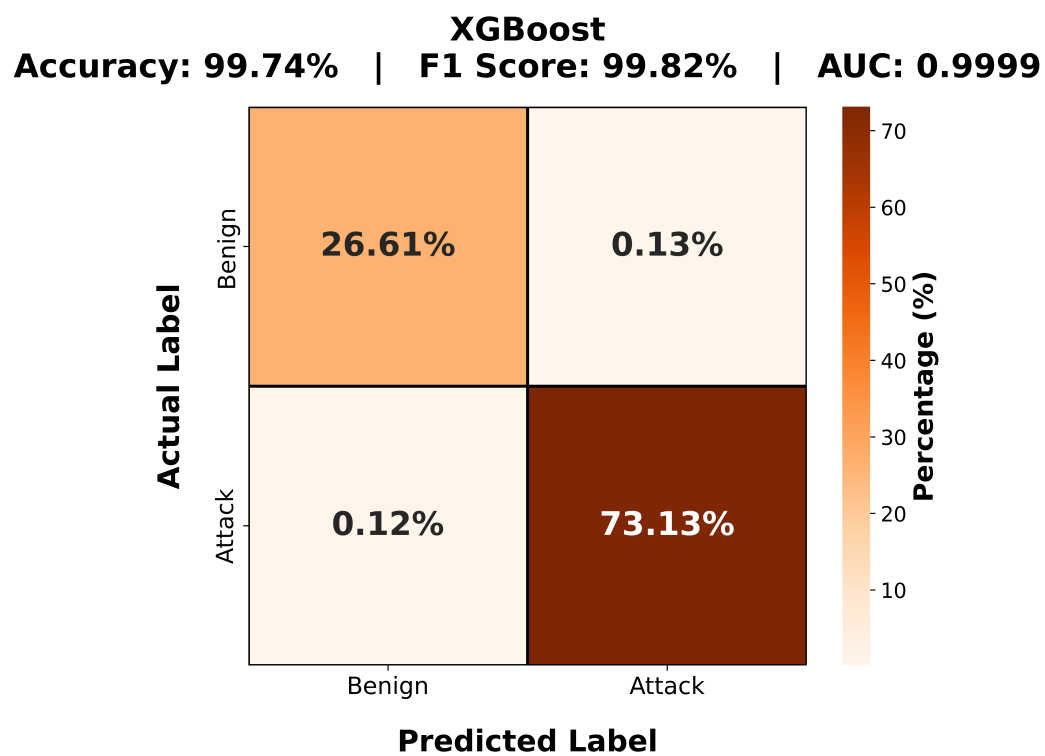


Figure 3. XGBoost binary confusion matrix.

Count. Because KNN has no training phase, all classification cost is incurred at inference time and scales linearly with the training set size, which limits its suitability for real-time deployment on resource-constrained V2X edge devices.

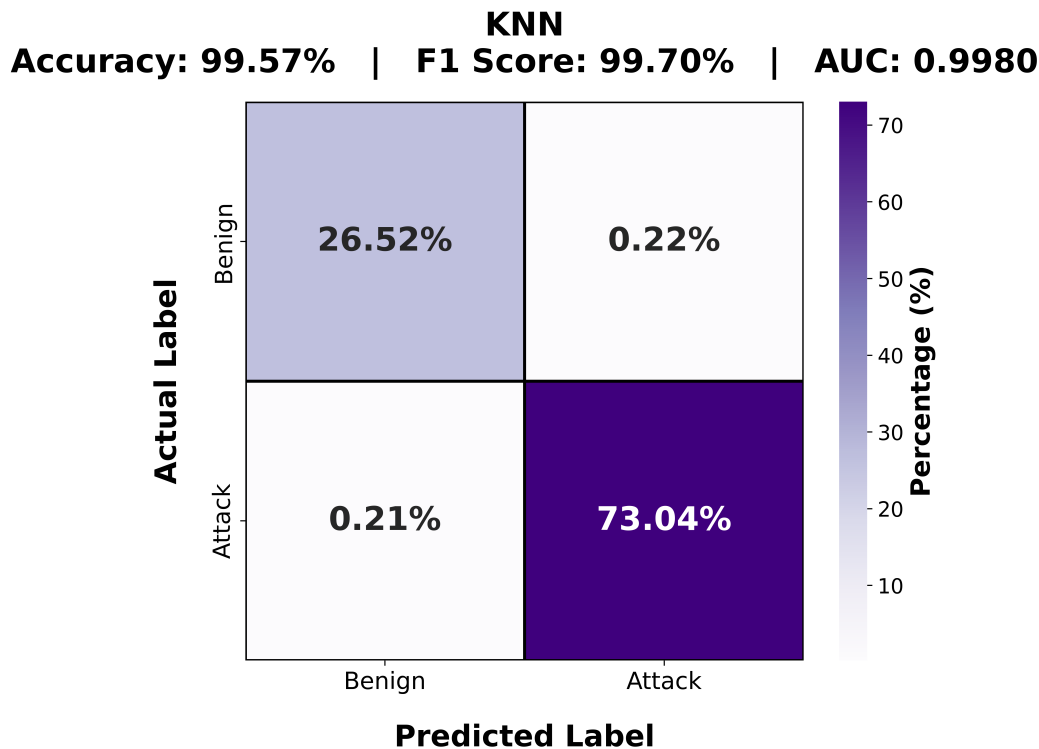


Figure 4. KNN binary confusion matrix.

1D CNN. The CNN (Fig. 5) correctly classifies 26.32% of flows as benign and 72.92% as attacks, with the highest false-positive rate (0.42%) and false-negative rate (0.34%) of all five models. Despite trailing on hard classification metrics, its AUC-ROC of 0.9990 indicates that its underlying probability ranking remains highly informative. The gap between AUC and threshold-based F1 performance is a calibration issue rather than a representational failure, and is analyzed further in Section 4.

3.2. ROC Analysis

Fig. 6 shows the ROC curves for all five models zoomed into the operationally relevant low false positive rate (FPR) region, which is the most meaningful range for deployment in safety-critical transportation systems where false alarms carry real operational cost. Random Forest and XGBoost reach essentially 100% true positive rate (TPR) at FPR < 0.01, demonstrating that both models can detect nearly all attacks before accumulating meaningful false positives. The Decision Tree requires approximately FPR ≈ 0.005 to reach 99% TPR, and KNN requires FPR ≈ 0.015, reflecting its sensitivity to low-importance features in the distance computation. The CNN, despite its lower hard-classification F1, maintains a competitive trajectory in this region consistent with its AUC of 0.9990. Fig. 7 summarizes the full AUC-ROC ranking, with XGBoost (0.999884) narrowly ahead of Random Forest (0.999859).

A practical implication is that XGBoost’s superior AUC combined with the Decision Tree’s slightly better F1 at the default 0.5 threshold reflects a well-known distinction: AUC measures discrimination across all thresholds, while F1 measures performance at a single threshold. XGBoost’s better probability calibration makes it the stronger choice for deployments that require adaptive threshold tuning based on operational safety requirements,

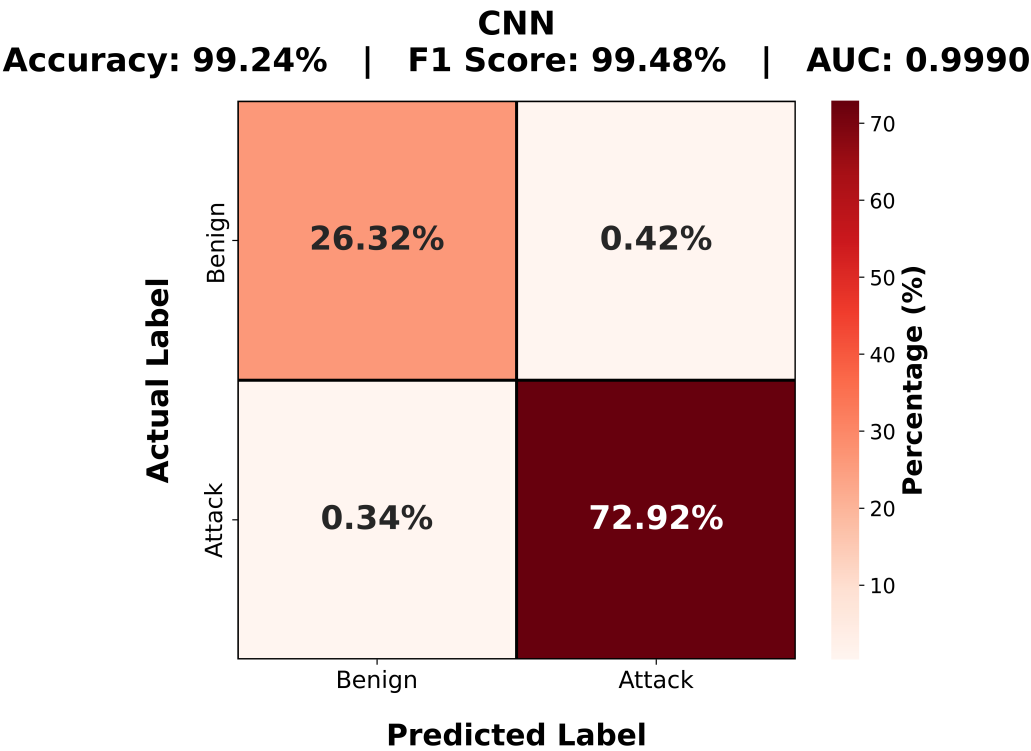


Figure 5. 1D CNN binary confusion matrix.

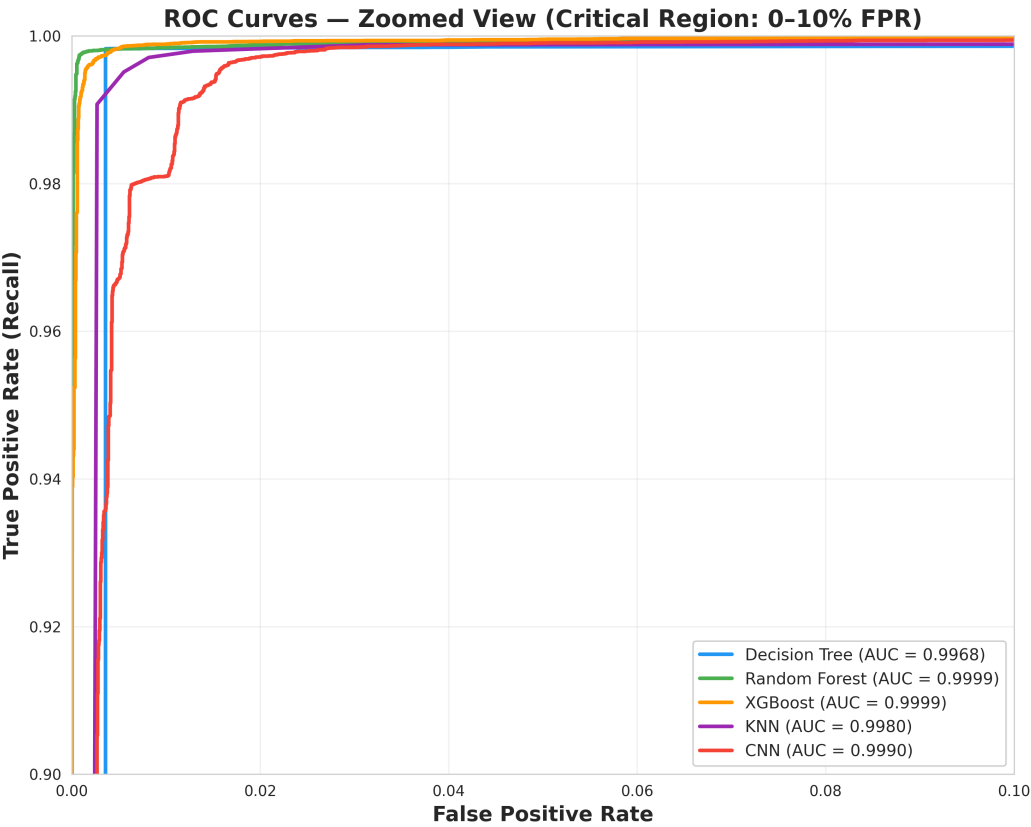


Figure 6. ROC curves zoomed into the low-FPR operating region for all five models.

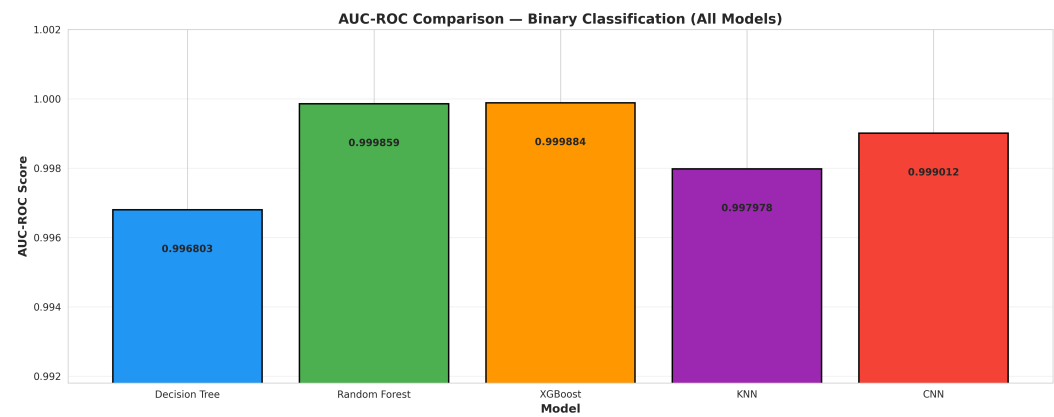


Figure 7. AUC-ROC ranking across all five models (binary task).

while the Decision Tree remains preferable when a fixed, interpretable decision boundary is needed.

3.3. Multi-Class Classification

Table 3 reports performance on the eleven-class attack-identification task, and Fig. 8 shows the corresponding confusion matrices.

Table 3. Multi-class classification results (11 attack classes).

Model	Acc.	Prec.	Rec.	F1	Time (min)
Decision Tree	99.56%	99.54%	99.56%	99.55%	0.12
XGBoost	99.52%	99.52%	99.52%	99.51%	0.54
Random Forest	99.43%	99.42%	99.43%	99.41%	0.31
KNN	99.24%	99.24%	99.24%	99.23%	—
1D CNN	98.34%	98.40%	98.34%	98.33%	0.89

A notable result is that the single Decision Tree outperforms both ensemble methods and the CNN on the multi-class task. Section 4 analyzes this outcome. A radar comparison across all metrics for both tasks is provided in Fig. 9.

3.4. Per-Class Performance

Table 4 reports per-class precision, recall, and F1-score for the best-performing multi-class model (Decision Tree). Most classes are detected with $F1 \geq 99.3\%$. Vulnerability Scan shows slightly reduced performance ($F1 = 95.77\%$) due to expected confusion with Port Scan, as both produce elevated RST flag counts and varied source ports. UDP Flood is a severe outlier ($F1 = 39.44\%$); its failure mode is analyzed in Section 4. Fig. 10 visualizes these results.

3.5. Feature Importance

Random Forest Gini-impurity feature importances (Fig. 11) reveal a striking concentration: only three packet-header features—RST Flag Count (28.34%), Forward Header Length (27.62%), and Source Port (23.69%)—jointly account for 79.65% of classification decisions. The remaining top-10 features (Bwd Packet Length Max, Fwd Packet Length Max, Connection Type, Fwd Seg Size Min, Flow IAT Min, Dst IP, Dst Port) each contribute less than 6%.

Because all three top features are obtainable from packet headers alone, no deep packet inspection (DPI) is required for accurate detection. This has direct deployment consequences: header-only inspection can be performed at line rate on resource-constrained

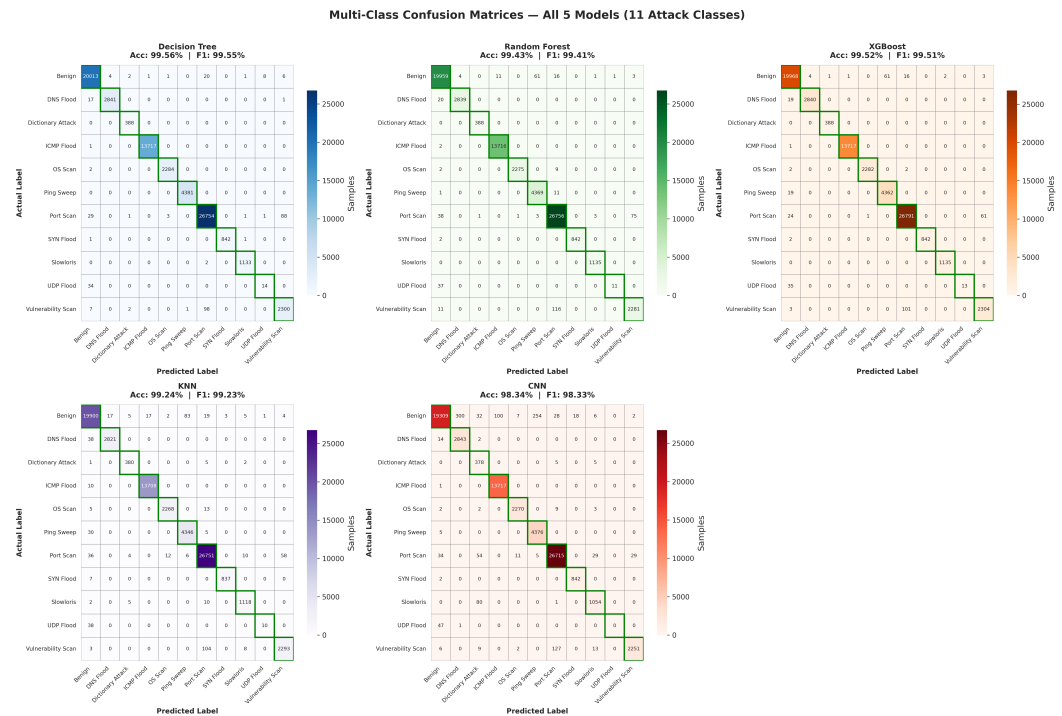


Figure 8. Multi-class confusion matrices for all five models across 11 attack classes.

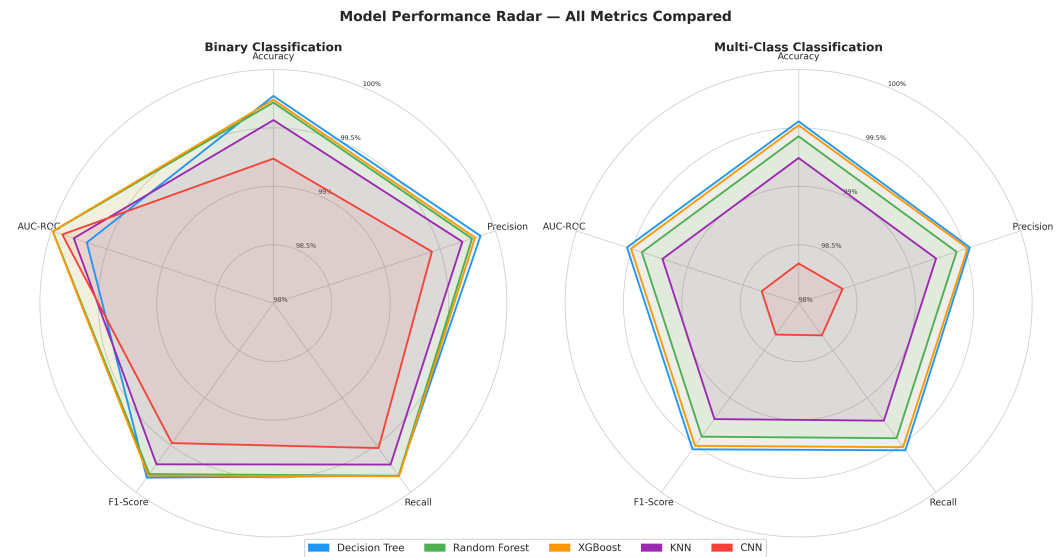
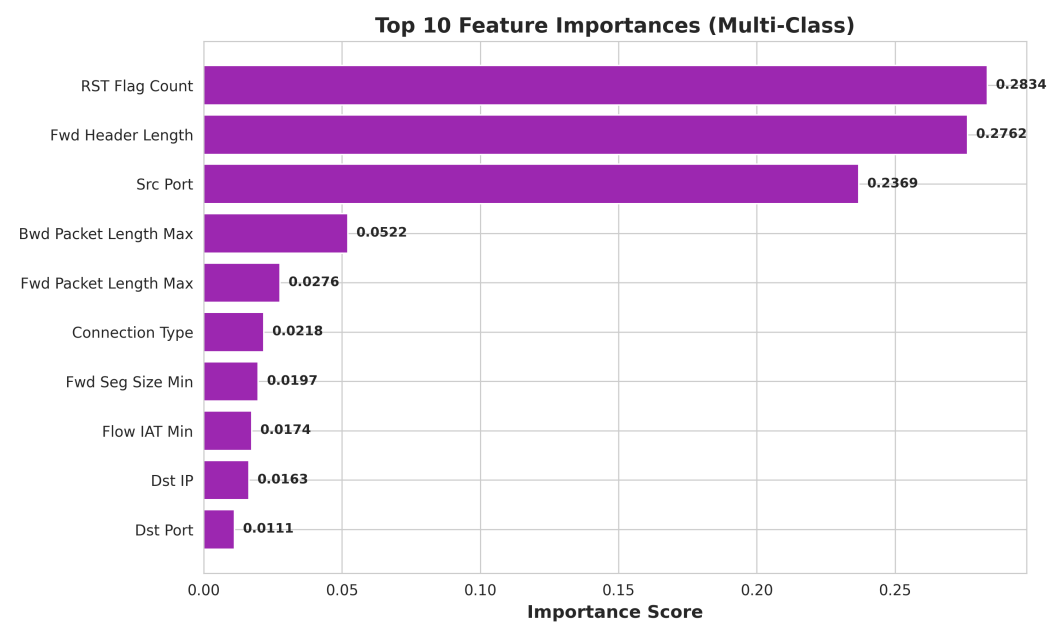


Figure 9. Radar chart comparing all five models across accuracy, precision, recall, F1, and AUC-ROC for both tasks.

Table 4. Per-class performance (Decision Tree, multi-class task).

Class	Prec.	Rec.	F1	Test N
ICMP Flood	99.99%	99.99%	99.99%	13,718
Ping Sweep	99.98%	100.00%	99.99%	4,381
SYN Flood	100.00%	99.76%	99.88%	844
OS Scan	99.83%	99.91%	99.87%	2,286
Slowloris	99.74%	99.82%	99.78%	1,135
Benign	99.55%	99.79%	99.67%	20,056
DNS Flood	99.86%	99.37%	99.61%	2,859
Port Scan	99.55%	99.54%	99.55%	26,877
Dictionary Attack	98.73%	100.00%	99.36%	388
Vulnerability Scan	96.03%	95.51%	95.77%	2,408
UDP Flood	60.87%	29.17%	39.44%	48

**Figure 10.** Per-class precision, recall, and F1-score for the Decision Tree on the multi-class task.**Figure 11.** Top-10 Random Forest feature importances (multi-class task).

V2X edge devices, and the model itself can be compact enough to fit within the memory and latency budget constraints of embedded vehicular hardware.

4. Discussion and Error Analysis

4.1. Why the CNN Underperforms

The 1D CNN achieves strong absolute accuracy (99.24% binary, 98.34% multi-class) but trails every classical model. Three mechanisms explain this outcome.

First, a Conv1D layer with kernel size 3 assumes that features at positions N , $N+1$, and $N+2$ are spatially correlated, as are adjacent pixels in an image. For ACI-IoT-2023, however, the 78 features are independently computed flow statistics arranged in an arbitrary column order, so the convolutional kernels learn correlations across meaningless neighbor groupings. This mismatch between inductive bias and data structure is consistent with recent tabular deep learning findings [15].

Second, the GlobalAveragePooling1D layer averages 19 post-convolution positions into a single 64-dimensional vector, diluting the signal from the single most predictive feature (RST Flag Count at 28.3% importance). Tree-based models, by contrast, can split on RST Flag Count alone at the root of the tree.

Third, at the default 0.5 sigmoid threshold, benign flows with legitimately elevated RST counts (from normal TCP teardown) produce sigmoid scores clustered near 0.52 and are misclassified. This is a threshold-calibration issue rather than a representational one: the CNN's AUC-ROC of 0.9990 ranks third overall, above KNN (0.9980) and the Decision Tree (0.9968), which indicates that its ranking of flows is informative even though its hard classifications at 0.5 are not optimal.

4.2. Why the Decision Tree Wins Multi-Class

Despite being the simplest model in the study, the Decision Tree achieves the highest multi-class F1-score (99.55%), outperforming both ensembles and the CNN. Two factors explain this result. First, the eleven attack classes have largely distinct signatures in the top-three feature space, which favors crisp axis-aligned decision boundaries over the averaged voting of an ensemble or the smoothed outputs of a softmax layer. Second, bootstrap sampling in Random Forest means that an average of roughly half of the 100 trees see no UDP Flood training samples (given only ~ 321 UDP Flood records in the 500,000-sample training set); those trees contribute no UDP Flood signal to the majority vote, which pushes UDP Flood flows toward larger classes. A single deterministic tree does not suffer this bootstrap penalty.

4.3. Class Imbalance and the UDP Flood Failure

UDP Flood recall is approximately 29% across all five models. The cause is identical in every case: only 791 total UDP Flood records exist in the raw dataset, producing only 48 test-set samples after stratified partitioning. No algorithm can learn a reliable decision boundary from this quantity of data. Importantly, the failure is a data problem rather than a model problem: applying a minority-class oversampler such as SMOTE [16] before splitting—for example, synthesizing $\sim 5,000$ UDP Flood records—would provide every model with a learnable signal without altering the evaluation protocol for majority classes. This is the most important single improvement we recommend for follow-up work.

4.4. Model-Specific Error Modes

Each model exhibits characteristic error modes beyond the shared UDP Flood failure:

- **Decision Tree:** Benign TCP teardowns with elevated RST counts occasionally cross the root split threshold and land in the attack subtree with no recovery path. Port Scan and Vulnerability Scan are confused at the exact Fwd Header Length boundary.
- **Random Forest:** Ensemble voting on borderline benign flows produces slim attack majorities (e.g., 60/40 splits) that the single Decision Tree routes more confidently.
- **XGBoost:** The learning rate of 0.1 causes later boosting rounds to overfit on hard borderline cases; a lower rate (0.01–0.05) with more estimators is likely to smooth these boundaries without harming overall accuracy.
- **KNN:** Uniform feature weighting gives RST Flag Count (28.3% importance) the same distance weight as Dst IP (1.6%), so flows can appear “close” to the wrong class purely from matching low-importance features. In addition, the overwhelming density of Port Scan in feature space systematically biases borderline flows toward a Port Scan label.

4.5. Deployment Implications

Three findings bear directly on deployment for IoT-enabled transportation networks:

Two-stage pipeline. XGBoost is recommended for the first-stage binary benign-vs-attack decision, based on its best-in-class AUC (0.9999) and fastest classical training time (0.08 min). A Decision Tree is recommended for the second-stage attack-type identification, based on its best-in-class multi-class F1 (99.55%) and interpretable decision rules.

Header-only inspection. Because 79.65% of classification decisions derive from three header-level features, deployment does not require DPI. This is critical for encrypted V2X channels and for edge devices where payload inspection is computationally or legally infeasible.

Edge-feasible retraining. Sub-minute classical training times (XGBoost at 0.08 min binary; Decision Tree at 0.12 min multi-class) support periodic on-device retraining as attack patterns evolve, without requiring cloud offload and without violating the latency and bandwidth constraints of V2V/V2I environments [17].

5. Conclusion

This paper presented a controlled comparison of five machine learning classifiers—Decision Tree, Random Forest, XGBoost, KNN, and a 1D CNN—for anomaly detection on the ACI-IoT-2023 dataset in the context of IoT-enabled transportation networks. All four classical models achieved greater than 99.5% accuracy on binary benign-versus-attack classification, with XGBoost producing the best AUC-ROC (0.9999) and the fastest classical training time (0.08 min). On the eleven-class attack-identification task, a single Decision Tree (99.56% accuracy, 99.55% F1) outperformed every ensemble and the CNN (98.34% F1), demonstrating that sharp, axis-aligned decision boundaries are well matched to tabular network flow statistics. Feature importance analysis showed that three packet-header features—RST Flag Count, Forward Header Length, and Source Port—account for 79.65% of all classification decisions, which enables lightweight real-time deployment on V2X edge devices without deep packet inspection.

The principal limitation identified in this study is class imbalance. UDP Flood recall of approximately 29% across all models is not an algorithmic failure but a direct consequence of only 48 test-set samples after stratified partitioning. Addressing this limitation through synthetic minority oversampling (SMOTE) [16] is the most important single improvement for future work. Additional directions include (i) evaluating lower learning rates and probability-calibrated objectives (`multi:softprob`) for XGBoost, (ii) integrating the two-stage XGBoost-then-Decision-Tree pipeline into a deployable V2X edge prototype, (iii) extending the evaluation to adversarial robustness, including the adversarial perception

attacks that motivated this work, and (iv) validating generalization across additional IoT traffic datasets beyond ACI-IoT-2023.

References

1. Dui, H.; Zhang, S.; Liu, M.; Dong, X.; Bai, G. IoT-Enabled Real-Time Traffic Monitoring and Control Management for Intelligent Transportation Systems. *IEEE Internet of Things Journal* **2024**, *11*, 15842–15854. <https://doi.org/10.1109/JIOT.2024.3351908>.
2. Sodhro, A.H.; Obaidat, M.S.; Abbasi, Q.H.; Pace, P.; Pirbhulal, S.; Yasar, A.U.H.; Fortino, G.; Imran, M.A.; Qaraqe, M. Quality of Service Optimization in an IoT-Driven Intelligent Transportation System. *IEEE Wireless Communications* **2019**, *26*, 10–17. <https://doi.org/10.1109/MWC.001.1900085>.
3. Bui, K.H.N.; Jung, J.J. ACO-Based Dynamic Decision Making for Connected Vehicles in IoT System. *IEEE Transactions on Industrial Informatics* **2019**, *15*, 5648–5655. <https://doi.org/10.1109/TII.2019.2906886>.
4. Song, Y.; Yu, F.R.; Zhou, L.; Yang, X.; He, Z. Applications of the Internet of Things (IoT) in Smart Logistics: A Comprehensive Survey. *IEEE Internet of Things Journal* **2021**, *8*, 4250–4274. <https://doi.org/10.1109/JIOT.2020.3034385>.
5. Wazid, M.; Singh, J.; Pandey, C.; Sherratt, R.S.; Das, A.K.; Giri, D.; Park, Y. Explainable Deep Learning-Enabled Malware Attack Detection for IoT-Enabled Intelligent Transportation Systems. *IEEE Transactions on Intelligent Transportation Systems* **2025**, *26*, 7231–7244. <https://doi.org/10.1109/TITS.2025.3525505>.
6. Ntafloukas, K.; Pasquale, L.; Martinez-Pastor, B.; McCrum, D.P. Identification of Vulnerable IoT Enabled Transportation Infrastructure into a Cyber-Physical Transportation Network. In Proceedings of the 2023 IEEE International Conference on Cyber Security and Resilience (CSR). IEEE, 2023, pp. 400–405.
7. Wazid, M.; Singh, J.; Pandey, C.; Sherratt, R.S.; Das, A.K.; Giri, D.; Park, Y. Explainable deep Learning-Enabled malware attack detection for IoT-Enabled intelligent transportation systems. *IEEE Transactions on Intelligent Transportation Systems* **2025**, *26*, 7231–7244.
8. He, Y.; Kong, M.; Du, C.; Yao, D.; Yu, M. Communication Security Analysis of Intelligent Transportation System Using 5G Internet of Things From the Perspective of Big Data. *IEEE Transactions on Intelligent Transportation Systems* **2023**, *24*, 2199–2207. <https://doi.org/10.1109/TITS.2022.3141788>.
9. Gyamfi, E.; Ansere, J.A.; Kamal, M.; Tariq, M.; Jurcut, A. An adaptive network security system for iot-enabled maritime transportation. *IEEE transactions on intelligent transportation systems* **2022**, *24*, 2538–2547.
10. Cheng, Q.; Hong, K.; Huang, K.; Liu, Z. Evaluating Effectiveness and Identifying Appropriate Methods for Anomaly Detection in Intelligent Transportation Systems. *IEEE Transactions on Intelligent Transportation Systems* **2025**.
11. Chen, C.; Liu, B.; Wan, S.; Qiao, P.; Pei, Q. An edge traffic flow detection scheme based on deep learning in an intelligent transportation system. *IEEE transactions on intelligent transportation systems* **2020**, *22*, 1840–1852.
12. Oseni, A.; Moustafa, N.; Creech, G.; Sohrabi, N.; Strelzoff, A.; Tari, Z.; Linkov, I. An explainable deep learning framework for resilient intrusion detection in IoT-enabled transportation networks. *IEEE Transactions on Intelligent Transportation Systems* **2022**, *24*, 1000–1014.
13. Nack, E.A.; McKenzie, M.C.; Bastian, N.D. ACI-IoT-2023: A Robust Dataset for Internet of Things Network Security Analysis. In Proceedings of the MILCOM 2024 - 2024 IEEE Military Communications Conference (MILCOM), 2024, pp. 1–6. <https://doi.org/10.1109/MILCOM61039.2024.10773916>.
14. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In Proceedings of the Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, New York, NY, USA, 2016; KDD '16, p. 785–794. <https://doi.org/10.1145/2939672.2939785>.

15. Shwartz-Ziv, R.; Armon, A. Tabular data: Deep learning is not all you need. *Information Fusion* **2022**, *81*, 84–90. <https://doi.org/https://doi.org/10.1016/j.inffus.2021.11.011>. 376 377
16. Chawla, N.V.; Bowyer, K.W.; Hall, L.O.; Kegelmeyer, W.P. SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research* **2002**, *16*, 321–357. 378 379
17. Chen, J.; Ran, X. Deep learning with edge computing: A review. *Proceedings of the IEEE* **2019**, *107*, 1655–1674. 380 381